

FUNDAMENTOS DE PROGRAMACÃO

Funções



Teach
3035

Índice

Array.....	03
Criando arrays.....	04
Acessando, alterando e adicionando elementos ao array utilizando o índice.....	04
Propriedades e métodos de arrays.....	05
Adicionar elementos.....	05
Remover elementos.....	06

Funções



Funções no JavaScript são blocos de código reutilizáveis e autocontidos que podem ser definidos e executados para realizar uma tarefa específica. Elas são uma parte fundamental da programação em JavaScript, permitindo que você agrupe instruções relacionadas em um único bloco lógico.

Declarando uma função



Uma função em JavaScript pode ter um nome, parâmetros opcionais e um bloco de código contendo as instruções a serem executadas quando a função é chamada. Em JavaScript, você pode declarar funções usando a sintaxe tradicional `function` ou a sintaxe de `arrow function` (`=>`).

- **Utilizando sintaxe tradicional**

- **Sintaxe**

```
function nomeDaFuncao(parametro1, parametro2) {  
  // Bloco de código com as instruções  
  return valorDeRetorno; // Opcional, se a função retorna um valor  
}
```

Declarando uma função



○ Exemplo:

```
function sum(a, b) {  
  return a + b;  
}
```

Nesse exemplo, temos uma função chamada `sum` que recebe dois parâmetros `a` e `b`. A função retorna a soma desses dois parâmetros usando a instrução `return`. Quando essa função é chamada e recebe valores para `a` e `b`, ela executa o bloco de código e retorna o resultado da soma.

● Utilizando sintaxe de arrow function:

A arrow function é caracterizada por uma seta (`=>`), os parâmetros ficam do lado esquerdo da seta e o corpo da função do lado direito. Ao executar, essa função terá o mesmo resultado que uma função escrita com a sintaxe tradicional.

○ Sintaxe

```
const nomeDaFuncao = (parametro1, parametro2) => {  
  // Bloco de código com as instruções  
  return valorDeRetorno; // Opcional, se a função retorna um valor  
}
```

Declarando uma função



○ Exemplo

```
function sum(a, b) {  
  return a + b;  
}
```

```
const sum = (a, b) => {  
  return a + b;  
}
```

Caso não sejam passados parâmetros na função, os parênteses ficam vazios.

○ Exemplo 2

```
() => 'Hello World!'
```

Comparando a Arrow Function com as outras funções, podemos notar algumas diferenças:

- Ela pode ser escrita em apenas 1 linha de código;
- Não precisa das palavras function e return;
- Não necessita o uso de chaves { }

Remover elementos



pop(): remove o último elemento do array e retorna o valor removido

```
cars.pop();  
console.log(cars) // saída: [Chevrolet, Volkswagen, Toyota, Renault]
```

shift(): remove o primeiro elemento do array

```
cars.shift();  
console.log(cars) // saída: [Volkswagen, Toyota, Renault]
```

slice(): retorna uma cópia superficial (shallow copy) de parte do array, especificada pelos índices de início e fim (fim não incluso).

```
let cars = [ "Fiat", "Chevrolet", "Volkswagen" ];  
let filteredCars = cars.slice(0,2);  
console.log(cars) // saída: [Fiat, Chevrolet]
```

Veja uma lista completa de todos os métodos clicando [aqui](#)

Chamando uma função



▼ Métodos

`Array.prototype[@@iterator]()`

`Array.prototype.at()`

`Array.prototype.concat()`

`Array.prototype.copyWithin()`

`Array.prototype.entries()`

`Array.prototype.every()`

`Array.prototype.fill()`

`Array.prototype.filter()`

`Array.prototype.find()`

`Array.prototype.findIndex()`

`Array.prototype.findLast()` (en-US)

`Array.prototype.findLastIndex()` (en-US)

`Array.prototype.flat()`

`Array.prototype.flatMap()`

`Array.prototype.forEach()`

`Array.from()`

`Array.fromAsync()` (en-US)

`Array.prototype.group()` (en-US) 

`Array.prototype.groupToMap()` (en-US)



`Array.prototype.includes()` :